Markdown Beginner Handbook

A Beginner Guide for Research Notes, READMEs, Math, Tables, Code, and Academic Workflows

Quick Reference + Worked Examples

2026

Contents

Before You Start: What Markdown Is **4**

1. Markdown Is Not One Single Standard **5**

2. Where You Can Write Markdown **5**
VS Code: the easiest local workflow . . . 5
GitHub: edit Markdown without installing anything . . . 6
HackMD: collaborative browser Markdown . . . 6
StackEdit: browser editor with live preview . . . 6
A privacy reminder . . . 6

3. Your First Markdown File **7**

Research Notes **7**
Next step . . . 7

4. Headings **7**
Source . . . 7
Rendered idea . . . 7

Level 1 heading **7**
Level 2 heading . . . 7

5. Paragraphs and Line Breaks **8**
Source . . . 8
Rendered . . . 8

6. Bold, Italic, and Strikethrough **8**
Source . . . 8
Rendered . . . 9

7. Markdown Characters You May Need to Escape **9**

8. Inline Code: One of the Most Useful Markdown Features **9**
Source . . . 9
Rendered . . . 10

9. Code Blocks **10**
Source . . . 10
Rendered . . . 10

10. Lists **11**
Unordered lists . . . 11
Ordered lists . . . 11

11. Nested Lists **12**

12. Task Lists / Checkboxes **12**

13. Links **12**
Named link . . . 12
Automatic URL . . . 12
Useful academic pattern . . . 13

14. Images **13**

15. Blockquotes **13**
Source . . . 13
Rendered . . . 13

16. Horizontal Rules **14**

17. Tables **14**
Source . . . 14
Rendered . . . 14

18. Pipe Characters Inside Tables **15**

19. Math in Markdown **15**
Inline math . . . 15
Display math . . . 15

20. Math: What Is Markdown and What Is LaTeX? **16**

21. Footnotes **16**
Source . . . 16
Rendered idea . . . 16

22. Comments That Do Not Render **16**

23. YAML Front Matter **17**

24. Academic Citations: Plain Markdown Does Not Standardize Them **17**

25. Cross-References: Also Tool-Dependent **18**

26. Linking to a Heading in the Same File **18**

27. Reference-Style Links **18**

28. Email Addresses and URLs **18**

29. File Paths and Commands **19**

30. A Good Research Project README Structure **19**

31. Paper Reading Note Template **20**

32. Research Log Template **21**

33. Seminar / Class Notes Template **21**

34. Data Dictionary in Markdown **22**

35. Literature Matrix in Markdown **22**

36. Details / Collapsible Sections (HTML Extension) **22**

37. HTML Inside Markdown **23**

38. Common Mistake: Using Markdown Like Microsoft Word **23**

39. Common Mistake: Too Many Heading Levels **23**

40. Common Mistake: Forgetting Blank Lines Around Blocks **23**

41. Common Mistake: Broken Code Fences **24**

42. Common Mistake: Backticks Inside Inline Code **24**

43. Common Mistake: Overusing Bold **24**

44. Common Mistake: Confusing a Hyphen with a List **25**

45. Common Mistake: Markdown Tables for Complex Regression Output **25**

46. Common Mistake: Assuming Math Works Everywhere **25**

47. Common Mistake: Assuming Citation Syntax Works Everywhere **25**

48. Common Mistake: Spaces in File Names **26**

49. Common Mistake: "final_final_v3_revised.md" **26**

50. GitHub README Habits for Research Repositories **26**

51. Useful README Badges: Optional, Not Necessary **26**

52. Markdown + Git **27**

53. Markdown + VS Code **27**

54. Markdown + Obsidian **27**

55. Markdown + Jupyter **27**

56. Markdown + Quarto **28**

57. Markdown + Pandoc **28**

58. When to Use Markdown vs. LaTeX **29**

59. When to Use Markdown vs. Word **29**

60. A Practical Research File Structure **29**

61. Research Meeting Note Template **30**

62. A Better Way to Write TODOs **31**

63. Reproducibility Notes **31**

64. Writing an Analysis Plan in Markdown **31**

65. Markdown Source vs. Rendered Output: Fast Mental Model **32**

66. Quick Cheat Sheet: Most-Used Syntax **33**
Text . . . 33
Structure . . . 33
Lists . . . 33
Links / images . . . 33
Quote . . . 33
Code . . . 33
Math . . . 33

67. Quick Cheat Sheet: "I Need To..." **33**

68. Quick Cheat Sheet: Common Mistakes **34**

69. What Beginners Should Learn First **34**

70. Recommended Beginner Workflow **35**

71. Final Template: Minimal Markdown Note **35**

72. Final Template: Paper Reading Note **36**

73. Final Template: Research Project README **36**

74. Final Advice **37**

Before You Start: What Markdown Is

Markdown is a **plain-text writing format**. You type ordinary text plus a small amount of punctuation, and a Markdown renderer turns it into formatted output such as HTML, a preview pane, or a PDF.

For students and researchers, Markdown is especially useful for:

- research notes and daily logs;
- reading notes for papers;
- project `README.md` files;
- replication instructions;
- seminar and class notes;
- data dictionaries;
- code documentation;
- GitHub repositories;
- Jupyter / VS Code / Obsidian notes;
- Quarto or Pandoc documents that combine prose, math, citations, and code.

Markdown is **not a replacement for LaTeX in every situation**. For a journal-style paper with complicated tables, theorem environments, or strict submission formatting, LaTeX is often better. Markdown is usually faster for notes, documentation, drafts, and reproducible workflows.

> **Beginner mental model:** Markdown is normal text first. Formatting is added with a few visible punctuation marks.
>
> **Want a fast lookup instead of a full lesson?** Start with `../05_Markdown_Cheat_Sheets/Markdown_Simple_Cheat_Sh` It covers the everyday basics in one short file. Use `../05_Markdown_Cheat_Sheets/Markdown_Advanced_Cheat_Sheet.pd` only when you need more detailed syntax, paths, tables, math, citations, GitHub patterns, or research-note templates.

---

1. Markdown Is Not One Single Standard

This is the most important fact beginners often miss.

There is a common core, but different systems add features.

Flavor / tool	Typical use	Important extras
CommonMark	general Markdown standard	basic syntax
GitHub Flavored Markdown (GFM)	GitHub READMEs/issues	tables, task lists, strikethrough
Pandoc Markdown	academic conversion	citations, footnotes, math, metadata
Quarto	research documents	citations, cross-references, executable code
Jupyter Markdown	notebooks	math, code-adjacent documentation
Obsidian	personal knowledge base	wiki links, callouts, embeds

A feature that works in Quarto may not work in a simple GitHub preview.

Rule: when something behaves strangely, first ask **which Markdown renderer am I using?**

---

2. Where You Can Write Markdown

You do not need a special Markdown program. A `.md` file is plain text, so the same file can move between local editors and browser-based tools.

For a non-technical beginner, these are the most practical options:

Tool	Installation?	Best use	Beginner note
VS Code	Yes	Local research notes, GitHub projects, documentation	Built-in Markdown preview; no Markdown extension required
GitHub web editor	No	READMEs and repository documentation	Edit `.md` files directly in the browser
HackMD	No	Collaborative notes and shared drafts	Real-time browser collaboration
StackEdit	No	Browser writing with live preview	Useful for practicing or quick Markdown documents
Jupyter / Colab	Depends	Notes next to code and analysis	Markdown appears inside text cells
Quarto	Depends	Reproducible academic documents	A good next step after ordinary Markdown

VS Code: the easiest local workflow

VS Code supports Markdown out of the box. Save a file with a `.md` extension and open the built-in preview.

On **macOS**:

```
Shift + Command + V     open preview
Command + K, then V     preview beside source
```

On **Windows/Linux**:

```
Ctrl + Shift + V        open preview
Ctrl + K, then V        preview beside source
```

You can also open the Command Palette and run:

```
Markdown: Open Preview
Markdown: Open Preview to the Side
```

A good beginner layout is:

```
left: notes.md source     right: rendered preview
```

For basic Markdown, you do **not** need an extension. Optional extensions such as `markdownlint` or Markdown All in One can be added later.

Official VS Code guide: https://code.visualstudio.com/Docs/languages/markdown

GitHub: edit Markdown without installing anything

GitHub renders Markdown in `README.md` files and elsewhere in its web interface. GitHub uses **GitHub Flavored Markdown (GFM)**.

A beginner can:

1. open a `.md` file in a repository;
2. click the edit/pencil control;
3. change the Markdown source;
4. preview the result;
5. commit/save the change.

Official GitHub guide: https://docs.github.com/en/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax

HackMD: collaborative browser Markdown

HackMD is useful for coauthor notes, seminars, reading groups, and shared outlines because multiple people can edit and discuss Markdown documents in the browser.

Website: https://hackmd.io/

StackEdit: browser editor with live preview

StackEdit is an in-browser Markdown editor with a live preview. Its website describes support for GitHub Flavored Markdown/CommonMark, mathematical expressions, and synchronization options.

Website: https://stackedit.io/

A privacy reminder

Browser tools are convenient, but research data and notes can be sensitive. Do not upload restricted data, confidential participant information, private API keys, or material prohibited by a data-use agreement.

For a longer step-by-step guide, see:

`04_Markdown_Handbooks/Markdown_Setup_and_Editors_Guide.md`

---

3. Your First Markdown File

A Markdown file usually ends in `.md`.

Example filename:

`research_notes.md`

A minimal file can be only:

```
# Research Notes

Today I cleaned the county-level employment data.

## Next step

Estimate the baseline specification.
```

Rendered:

Research Notes

Today I cleaned the county-level employment data.

Next step

Estimate the baseline specification.

You do **not** need a document class, package declarations, or `\begin{document}` as you do in LaTeX.

---

4. Headings

Use `#` characters at the beginning of a line.

Source

```
# Level 1 heading
## Level 2 heading
### Level 3 heading
#### Level 4 heading
```

Rendered idea

Level 1 heading

Level 2 heading

Level 3 heading

Level 4 heading For research notes, a practical hierarchy is:

```
# Project title
## Data
### Raw data
### Cleaning
## Empirical strategy
## Results
## Questions
```

Common mistake: no space after #

Prefer:

```
## Results
```

not:

```
##Results
```

---

5. Paragraphs and Line Breaks

Markdown uses **blank lines** to separate paragraphs.

Source

```
This is the first paragraph.

This is the second paragraph.
```

Rendered

This is the first paragraph.

This is the second paragraph.

A normal single line break in the source often **does not** create a new paragraph.

```
Line one.
Line two.
```

may render as one paragraph depending on the renderer.

If you truly need a hard line break, common options are:

```
Line one.
Line two.
```

(two spaces at the end of the first line), or in HTML-capable Markdown:

```
Line one.<br>
Line two.
```

For academic prose: usually use blank lines for paragraphs instead of forcing line breaks.

---

6. Bold, Italic, and Strikethrough

Source

```
**bold text**
*italic text*
***bold and italic***
~~deleted or obsolete text~~
```

Rendered

bold text
italic text
bold and italic
~~deleted or obsolete text~~

Useful research-note pattern:

```
**Finding:** The coefficient is stable across specifications.

**Question:** Why does the estimate fall after adding region fixed effects?

**TODO:** Check whether the 2012 observations are duplicated.
```

Rendered:

Finding: The coefficient is stable across specifications.

Question: Why does the estimate fall after adding region fixed effects?

TODO: Check whether the 2012 observations are duplicated.

---

7. Markdown Characters You May Need to Escape

Markdown punctuation has meaning. To show it literally, prepend a backslash.

Want to display	Often type
*	*
_	_
#	\#
[	\[
]	\]
`	\`
>	\>
-	\- when needed
\	\\ in some contexts

Example:

```
The variable is named `income_after_tax`.
```

For filenames, variable names, commands, and literal punctuation, **inline code** is usually cleaner than escaping everything.

---

8. Inline Code: One of the Most Useful Markdown Features

Use single backticks to show code or literal text.

Source

```
Run `reg y x, robust` after cleaning the sample.
The main file is `analysis.R`.
The variable is `county_id`.
```

Rendered

Run `reg y x, robust` after cleaning the sample.
The main file is `analysis.R`.
The variable is `county_id`.

This is the Markdown equivalent of the "show the command literally" idea you used in the LaTeX guide.

It is excellent for:

- filenames;
- variable names;
- shell commands;
- R / Python / Stata commands;
- LaTeX commands such as `\qquad`;
- package names;
- directory names.

---

9. Code Blocks

Use triple backticks for a block of code.

Source

````
```r
model <- lm(y ~ x1 + x2, data = df)
summary(model)
```
````

Rendered

```r
model <- lm(y ~ x1 + x2, data = df)
summary(model)
```

The word after the opening backticks is the **language identifier**. Many renderers use it for syntax highlighting.

Common examples for academic and research work:

````
```python
import pandas as pd
```

```r
library(fixest)
```

```stata
reg y x1 x2, robust
```

```bash
pandoc notes.md -o notes.pdf
```

```latex
\begin{align}
y_i &= x_i'\beta + \varepsilon_i.
````
````

```
\end{align}
```
```

### Code block without a language

```
```
plain text
or pseudocode
```
```

### Common mistake: forgetting to close the fence

Every opening triple backtick needs a closing triple backtick.

---

# 10. Lists

## Unordered lists

```
- Download raw data
- Clean identifiers
- Merge county controls
- Estimate baseline model
```

Rendered:

- Download raw data
- Clean identifiers
- Merge county controls
- Estimate baseline model

You can also use `*`, but using `-` consistently is simple.

Ordered lists

```
1. Load the data.
2. Restrict the sample.
3. Estimate the model.
4. Export the table.
```

Rendered:

1. Load the data.
2. Restrict the sample.
3. Estimate the model.
4. Export the table.

Useful trick: you can type `1.` repeatedly

```
1. First step
1. Second step
1. Third step
```

Many Markdown renderers automatically display 1, 2, 3.

This makes reordering items easier.

```

11. Nested Lists

Indent sub-items.

```
- Main analysis
  - Baseline sample
  - Alternative sample
- Robustness
  - Different controls
  - Different clustering
```

Rendered:

- Main analysis
  - Baseline sample
  - Alternative sample
- Robustness
  - Different controls
  - Different clustering

For readability, use consistent indentation, commonly two or four spaces depending on your editor/renderer.

---

12. Task Lists / Checkboxes

Very useful for research workflows.

```
- [x] Download CPS data
- [x] Construct treatment variable
- [ ] Check missing values
- [ ] Reproduce Figure 2
- [ ] Draft data appendix
```

Rendered in supporting systems such as GitHub:

- ☒ Download CPS data
- ☒ Construct treatment variable
- ☐ Check missing values
- ☐ Reproduce Figure 2
- ☐ Draft data appendix

This is typically associated with **GitHub Flavored Markdown**.

---

13. Links

Named link

```
[American Economic Review](https://www.aeaweb.org/journals/aer)
```

Rendered:

American Economic Review

Automatic URL

```
<https://www.aeaweb.org/>
```

Rendered:

https://www.aeaweb.org/

Useful academic pattern

```
- Paper: [Title](https://example.com/paper.pdf)
- Data: [Replication files](https://example.com/data)
- Code: [GitHub repository](https://github.com/example/repo)
```

---

14. Images

Basic syntax:

```
![Alternative text](figures/main_result.png)
```

The syntax is almost the same as a link, but starts with `!`.

- `Alternative text` describes the image.
- `figures/main_result.png` is the image path.

Recommended project structure:

```
project/
|-- README.md
|-- notes/
|-- code/
|-- data/
`-- figures/
    `-- main_result.png
```

Then your README can use:

```
![Main result](figures/main_result.png)
```

Common mistake: wrong relative path

If the Markdown file is in `notes/` but the figure is in the project-level `figures/` folder, you may need:

```
![Main result](../figures/main_result.png)
```

---

15. Blockquotes

Use `>`.

Source

```
> The identifying assumption is most credible when treatment timing is unrelated to short-run
 ↪  local shocks.
```

Rendered

> The identifying assumption is most credible when treatment timing is unrelated to short-run local shocks.

Useful for:

- quotations from a paper;

- advisor comments;
- your own “important warning” notes;
- excerpts you want visually separated from your summary.

Nested blockquote:

```
> Main quote
>
>> Your reaction to the quote
```

---

16. Horizontal Rules

Use three or more hyphens:

```
---
```

Rendered:

Useful for separating sections in notes or READMEs, but do not overuse them if headings already provide structure.

---

17. Tables

Markdown tables are ideal for **small, simple tables**.

Source

```
| Variable | Mean | SD |
|---|---:|---:|
| Income | 52.4 | 13.2 |
| Employment | 0.81 | 0.39 |
| Age | 41.7 | 11.8 |
```

Rendered

Variable	Mean	SD
Income	52.4	13.2
Employment	0.81	0.39
Age	41.7	11.8

Alignment syntax:

```
:---   left aligned
:---:  centered
---:   right aligned
```

Good use cases

- variable dictionaries;
- small summary tables;
- project status tables;
- literature comparison tables;

- file descriptions.

Poor use cases

Markdown tables are not ideal for a complicated journal regression table with panels, multicolumn headings, notes, and precise spacing. Use LaTeX, HTML, or generated tables for those.

---

18. Pipe Characters Inside Tables

A `|` separates table columns. If you need a literal pipe, escape it when the renderer supports this:

```
A \| B
```

or put the content in inline code when appropriate.

This is a common reason Markdown tables suddenly gain an extra column.

---

19. Math in Markdown

Math support is **renderer-dependent**. GitHub, the built-in VS Code preview, Pandoc, Quarto, Jupyter, and many note systems support LaTeX-style math through MathJax/KaTeX or a TeX engine. Some simpler viewers show the dollar signs literally.

Inline math

```
The sample mean is $\bar{x} = n^{-1}\sum_{i=1}^n x_i$.
```

Rendered here:

The sample mean is $\bar{x} = n^{-1}\sum_{i=1}^n x_i$.

Display math

```
$$
y_i = x_i'\beta + \varepsilon_i.
$$
```

Rendered:

$$y_i = x_i'\beta + \varepsilon_i.$$

Useful fact for your LaTeX knowledge

Inside a math-capable Markdown renderer, familiar LaTeX math commands often work:

```
$$
y_i = x_i'\beta + \varepsilon_i,
\qquad i=1,\ldots,n.
$$
```

So commands such as `\quad`, `\qquad`, `\frac`, `\sum`, `\mathbb`, and `\text` may be usable **inside math**, even though Markdown itself is not LaTeX.

---

20. Math: What Is Markdown and What Is LaTeX?

This distinction matters.

```
**bold**
```

is Markdown syntax.

But:

```
\qquad
\frac{a}{b}
\sum_{i=1}^n
```

are LaTeX math commands interpreted by a math renderer.

Think of it as:

> Markdown handles document structure and simple formatting; a math engine handles the formula inside `$...$` or `$$...$$`.

---

21. Footnotes

Footnote syntax is not part of the smallest Markdown core, but Pandoc, GitHub, and many academic tools support it.

Source

```
The sample excludes observations with missing wages.[^1]

[^1]: Results are similar when these observations are retained and missing wages are coded
↪  separately.
```

Rendered idea

The sample excludes observations with missing wages.[1]

You can keep the footnote definition later in the file instead of interrupting the paragraph.

---

22. Comments That Do Not Render

HTML comments are widely useful:

```
<!-- TODO: verify sample size before sending to advisor. -->
```

The comment remains in the source but normally does not appear in rendered output.

Excellent for private drafting notes:

```
<!-- Ask Alex whether we should cluster at the county or state level. -->
```

---

[1] Results are similar when these observations are retained and missing wages are coded separately.

23. YAML Front Matter

Pandoc, Quarto, R Markdown, and other systems can read metadata at the very top of a file.

```yaml
---
title: "Labor Market Project Notes"
author: "Your Name"
date: "2026-09-26"
---
```

This is called **YAML front matter**.

In a Quarto/Pandoc workflow you may add options such as:

```yaml
---
title: "Research Notes"
author: "Your Name"
format: pdf
toc: true
number-sections: true
---
```

Important: exact option names differ across tools. Do not assume every YAML option works in every Markdown renderer.

---

24. Academic Citations: Plain Markdown Does Not Standardize Them

Plain Markdown does not define a universal academic citation system.

Pandoc / Quarto commonly support citation keys such as:

```
Acemoglu and Autor emphasize task-based changes in labor demand [@example2020].

Related work reaches similar conclusions [@example2020; @other2022].
```

A bibliography file might be specified in YAML:

```yaml
---
bibliography: references.bib
---
```

A BibTeX entry could look like:

```bibtex
@article{example2020,
  author  = {Author, Alice and Author, Bob},
  title   = {Example Paper},
  journal = {Example Journal},
  year    = {2020},
  volume  = {10},
  pages   = {1--25}
}
```

This is **Pandoc/Quarto citation syntax**, not universal Markdown.

---

25. Cross-References: Also Tool-Dependent

Basic Markdown has links but no universal system for "Figure 2" or "Equation 3" references.

Quarto adds powerful cross-reference syntax. For example, a figure may have a label such as `fig-main`, and the prose can refer to `@fig-main`.

Because syntax varies by tool, treat cross-referencing as a feature of **Quarto/Pandoc extensions**, not core Markdown.

For simple notes, ordinary headings and links are often enough.

---

26. Linking to a Heading in the Same File

Many renderers automatically generate anchors for headings.

For a heading:

```
## Tables
```

you may be able to link with:

```
[Jump to tables](#tables)
```

Rendered: a clickable link that scrolls to the "Tables" heading of the same file.

Anchor-generation rules can vary, especially for punctuation and duplicate headings.

---

27. Reference-Style Links

Useful when the same URL appears repeatedly or you want cleaner prose.

```
Read the [AER article][aer-paper] and the [replication files][replication].

[aer-paper]: https://example.com/paper
[replication]: https://example.com/data
```

This keeps long URLs away from the paragraph.

---

28. Email Addresses and URLs

You can often use:

```
<name@example.edu>
```

or:

```
<https://example.edu/project>
```

For a named link, prefer:

```
[Project page](https://example.edu/project)
```

---

29. File Paths and Commands

Use inline code:

```
Open `data/raw/cps_2025.csv`.
Run `code/01_clean.R` before `code/02_estimate.R`.
```

This is cleaner than italics or quotation marks because it immediately signals a literal filename/path.

---

30. A Good Research Project README Structure

A useful `README.md` might look like this:

````
# Project Title

One-sentence description of the project.

## Repository structure

```text
project/
|-- code/
|-- data/
|-- figures/
|-- output/
`-- README.md
```

## Data

- `data/raw/`: original files; never edited manually
- `data/clean/`: analysis-ready data

## Replication order

1. Run `code/01_clean.R`.
2. Run `code/02_analysis.R`.
3. Run `code/03_figures.R`.

## Main outputs

- `output/table1.tex`
- `figures/figure1.pdf`

## Software

- R 4.x
- Stata 19

## Notes

Large confidential data are not included in the repository.
````

This is one of the highest-value uses of Markdown in research.

---

31. Paper Reading Note Template

```
# Paper Title

**Authors:**
**Journal / year:**
**Link:**
**Date read:**

## One-sentence contribution

What is the paper's main contribution?

## Research question

What exactly is being asked?

## Data

- Unit of observation:
- Sample period:
- Main outcome:
- Main explanatory variable:

## Empirical / theoretical approach

Explain the approach in your own words.

## Main result

What should you remember six months from now?

## Identification / key assumptions

1.
2.

## What I learned

-
-

## Questions / weaknesses

- [ ]
- [ ]

## Useful citation / quote

> Short quotation or paraphrase location.

## Connection to my work

Why does this paper matter for my project?
```

32. Research Log Template

A daily research log is extremely useful during long research projects.

```
# Research Log - 2026-09-26

## Goal for today

Reproduce the baseline summary statistics.

## What I did

- Cleaned county identifiers.
- Removed duplicate observations.
- Re-ran `01_clean.R`.

## Result

The final sample contains **18,427 observations**.

## Problem encountered

Merge rate fell from 98% to 91% after adding 2011 data.

## Hypothesis

County FIPS codes may have lost leading zeros.

## Next actions

- [ ] Inspect unmatched FIPS codes.
- [ ] Compare raw and cleaned identifiers.
- [ ] Re-run Table 1.

## Files changed

- `code/01_clean.R`
- `notes/data_issues.md`
```

This kind of documentation is often more valuable than trying to remember what you did two weeks later.

---

33. Seminar / Class Notes Template

```
# Seminar: Speaker - Paper Title

**Date:**
**Speaker:**
**Institution:**

## Research question

## Why it matters

## Model / empirical design
```

```
## Data

## Main result

## Questions from the audience

1.
2.

## My questions

-
-

## Ideas relevant to my research

-
```

---

34. Data Dictionary in Markdown

Markdown tables work well for a lightweight data dictionary.

```
| Variable | Type | Description | Source |
|---|---|---|---|
| `county_fips` | string | 5-digit county FIPS | Census |
| `employment` | numeric | employment rate | CPS |
| `treatment` | binary | treatment indicator | constructed |
```

Rendered:

Variable	Type	Description	Source
`county_fips`	string	5-digit county FIPS	Census
`employment`	numeric	employment rate	CPS
`treatment`	binary	treatment indicator	constructed

---

35. Literature Matrix in Markdown

For a small literature set:

```
| Paper | Question | Data | Method | Main takeaway |
|---|---|---|---|---|
| Author A (2024) | ... | ... | ... | ... |
| Author B (2025) | ... | ... | ... | ... |
```

For dozens or hundreds of papers, a spreadsheet or reference manager is usually better.

---

36. Details / Collapsible Sections (HTML Extension)

On renderers that permit HTML:

```
<details>
<summary>Show robustness notes</summary>

Additional details go here.

</details>
```

This is useful in GitHub READMEs, but it is **not universal Markdown** and may behave differently in PDF conversion.

---

37. HTML Inside Markdown

Many Markdown engines allow some raw HTML:

```
<br>
<sup>1</sup>
<kbd>Ctrl</kbd> + <kbd>Enter</kbd>
```

Use HTML only when Markdown syntax is insufficient. Heavy HTML reduces portability.

---

38. Common Mistake: Using Markdown Like Microsoft Word

Markdown is designed around **structure**, not manual visual placement.

Avoid trying to create layouts with dozens of spaces:

```
Name: John                                  Date: 2026
```

Use a table, a proper renderer feature, or another format if alignment is important.

Likewise, do not use repeated blank lines to push text down a page.

---

39. Common Mistake: Too Many Heading Levels

A research note rarely needs:

```
###### Level 6 heading
```

Prefer a shallow hierarchy:

```
# Project
## Main section
### Subsection
```

If you need six levels, the note may need restructuring.

---

40. Common Mistake: Forgetting Blank Lines Around Blocks

Some renderers are forgiving; others are not.

Safer:

```
Paragraph text.

- item one
- item two

Next paragraph.
```

instead of tightly attaching every block.

---

41. Common Mistake: Broken Code Fences

Wrong:

````
```r
model <- lm(y ~ x, data = df)

Next paragraph begins here...
````

The closing fence is missing, so the rest of the document may become code.

Correct:

````
```r
model <- lm(y ~ x, data = df)
```

Next paragraph begins here.
````

---

42. Common Mistake: Backticks Inside Inline Code

If your literal text itself contains a backtick, use more backticks as the outer delimiter.

Example source:

```
``Use `code` inside this literal text.``
```

The exact behavior depends on the Markdown parser, but the general principle is: **use a longer backtick delimiter than the backticks inside the content**.

---

43. Common Mistake: Overusing Bold

If every sentence is bold, nothing is emphasized.

Better:

```
**Main finding:** Employment rises by roughly 3 percent.

The result is stable across the preferred specifications.
```

Use bold as a signal, not as the default typography.

```

44. Common Mistake: Confusing a Hyphen with a List

A hyphen at the start of a line followed by a space creates a list item:

```
- item
```

If you mean literal text, put it in inline code or escape it if needed.

---

45. Common Mistake: Markdown Tables for Complex Regression Output

A table such as:

```
Panel A
Columns (1)-(8)
multicolumn groupings
standard errors beneath coefficients
multiple notes
```

will become painful in plain Markdown.

Better choices:

- LaTeX table;
- HTML table;
- generated output from R/Stata/Python;
- Quarto table tools.

Markdown is best for **small human-readable tables**, not precision journal typesetting.

---

46. Common Mistake: Assuming Math Works Everywhere

This:

```
$E[Y_i \mid X_i]$
```

works in many renderers, but not in every basic Markdown preview.

If math matters, use a known math-capable environment such as:

- GitHub (README and `.md` files);
- the built-in VS Code Markdown preview;
- Quarto;
- Jupyter;
- Pandoc with math support;
- Obsidian.

---

47. Common Mistake: Assuming Citation Syntax Works Everywhere

This:

```
[@smith2025]
```

is excellent in Pandoc/Quarto, but GitHub will usually display it literally.

Again: know the renderer.

---

48. Common Mistake: Spaces in File Names

This is legal on modern systems but often annoying in code and reproducible workflows.

Prefer:

```
main_results.md
paper_notes_2026-09-26.md
figure_01.png
```

instead of:

```
Main Results Final New.md
```

For reproducible research, consistent filenames matter more than fancy filenames.

---

49. Common Mistake: "final_final_v3_revised.md"

Version numbers in filenames become chaotic.

Better:

- Git for version history;
- clear stable filenames;
- dated research logs when chronology matters.

Example:

```
README.md
analysis_plan.md
2026-09-26_research_log.md
```

---

50. GitHub README Habits for Research Repositories

A useful research repository README should answer:

1. What is this project?
2. What data are required?
3. Where are files located?
4. In what order should scripts run?
5. What software/packages are needed?
6. What files are generated?
7. Are any data restricted?
8. Who should be contacted with questions?

Markdown is nearly ideal for this job.

---

51. Useful README Badges: Optional, Not Necessary

GitHub projects sometimes include badges for build status, DOI, license, etc.

These are useful for public software/data projects but usually unnecessary for a private dissertation repository.

Do not confuse professional documentation with decorative badges.

---

52. Markdown + Git

Markdown works beautifully with Git because `.md` files are plain text.

Git can show exactly which lines changed.

Typical workflow:

```
git status
git add README.md
git commit -m "Update replication instructions"
git push
```

This is a major advantage over binary formats such as `.docx` for technical project documentation.

---

53. Markdown + VS Code

Typical VS Code workflow:

1. Create `notes.md`.
2. Write Markdown in the editor.
3. Open Markdown Preview.
4. Edit source and watch the preview update.

Useful built-in concept:

`source pane | rendered preview`

This is one of the easiest ways for a beginner to learn Markdown because every change is immediately visible.

---

54. Markdown + Obsidian

Obsidian uses Markdown files for a personal knowledge base.

Useful research applications:

- one note per paper;
- one note per research idea;
- seminar notes;
- advisor-meeting notes;
- linking related concepts.

Obsidian adds features such as `[[wiki links]]` that are **not universal Markdown**.

---

55. Markdown + Jupyter

Jupyter notebooks alternate between:

- **Markdown cells** for explanation; and
- **code cells** for execution.

A good empirical notebook should not be 100 lines of unexplained code. Use Markdown cells to explain:

- what the next code block does;

- why the transformation is necessary;
- what the result means;
- what remains unresolved.

This makes notebooks much easier to revisit months later.

---

56. Markdown + Quarto

Quarto is particularly useful for research because it combines Markdown-style writing with executable code, citations, equations, cross-references, and multiple outputs.

Conceptually:

```
Markdown prose
+ equations
+ R/Python/Julia code
+ bibliography
+ figures/tables
        ↓
HTML / PDF / Word / slides
```

A minimal Quarto document may start:

````
---
title: "Replication Notes"
format: html
---

# Analysis

```{r}
summary(df)
```
````

Quarto has its own syntax and options, so treat it as a **Markdown-based publishing system**, not plain Markdown.

---

57. Markdown + Pandoc

Pandoc converts between many document formats.

Example:

```
pandoc notes.md -o notes.pdf
```

or:

```
pandoc notes.md -o notes.docx
```

Academic features can include:

- math;
- citations;
- bibliography;
- footnotes;
- metadata;
- PDF conversion through LaTeX.

This is useful when you like writing in Markdown but need a shareable PDF or Word document.

---

58. When to Use Markdown vs. LaTeX

Task	Markdown	LaTeX
quick research notes	excellent	unnecessary
project README	excellent	poor fit
replication instructions	excellent	unnecessary
literature notes	excellent	good but slower
seminar notes	excellent	slower
GitHub documentation	excellent	poor fit
simple equations	good if renderer supports math	excellent
complicated equations	okay	excellent
polished regression tables	limited	excellent
theorem-heavy theory paper	limited	excellent
journal submission	sometimes	usually excellent
reproducible Quarto report	excellent	backend may still use LaTeX

A productive research workflow often uses **both**, not one or the other.

---

59. When to Use Markdown vs. Word

Markdown is better when:

- you want plain-text version control;
- you work with code;
- structure matters more than manual layout;
- you want fast notes;
- you use GitHub/Quarto/Jupyter.

Word is better when:

- collaborators require track changes;
- complex visual page layout is needed;
- nontechnical coauthors prefer WYSIWYG editing;
- the final deliverable must be `.docx`.

Pandoc/Quarto can sometimes bridge the two.

---

60. A Practical Research File Structure

```
project_name/
|-- README.md
|-- paper/
|   |-- main.tex
|   `-- references.bib
|-- notes/
|   |-- literature/
|   |-- meetings/
|   `-- research_log/
```

```
|-- code/
|   |-- 01_clean.R
|   |-- 02_analysis.R
|   `-- 03_figures.R
|-- data/
|   |-- raw/
|   `-- clean/
|-- figures/
`-- output/
```

Markdown belongs naturally in:

```
README.md
notes/literature/*.md
notes/meetings/*.md
notes/research_log/*.md
```

LaTeX can remain in `paper/`.

---

61. Research Meeting Note Template

```
# Advisor Meeting - 2026-09-26

## Updates since last meeting

- Completed baseline sample construction.
- Replicated Table 1.
- Added county-level controls.

## Results to discuss

1. Baseline coefficient is stable.
2. Standard errors increase substantially with state-level clustering.

## Questions

- Which clustering level is preferred?
- Should 2010 be excluded because of the coding change?

## Decisions

- Keep 2010 for now.
- Add state-specific trends only as a robustness check.

## Next actions

- [ ] Run alternative clustering.
- [ ] Create event-time figure.
- [ ] Draft data section.

## Next meeting

TBD.
```

---

62. A Better Way to Write TODOs

Instead of scattering vague notes:

```
fix this later
```

use searchable tags:

```
**TODO:** Check sample restriction.

**VERIFY:** Confirm Table 2 observation count.

**QUESTION:** Why does the coefficient change after 2018?

**DECISION:** Use county-level clustering in the main table.
```

Then your editor can search `TODO:` or `VERIFY:` across the project.

---

63. Reproducibility Notes

A good Markdown replication note might include:

```
## Environment

- R 4.5.x
- Stata 19
- Python 3.13

## Execution order

1. `01_download.R`
2. `02_clean.R`
3. `03_analysis.R`
4. `04_tables.R`

## Expected runtime

Approximately 12 minutes on a laptop.

## Restricted data

The confidential administrative dataset is not distributed with the repository.
```

This is much more useful to a future collaborator than undocumented code.

---

64. Writing an Analysis Plan in Markdown

```
# Analysis Plan

## Research question

## Sample

### Inclusion criteria
```

```
### Exclusion criteria

## Outcomes

### Primary outcome

### Secondary outcomes

## Main specification

$$
y_i = x_i'\beta + \varepsilon_i.
$$

## Standard errors

## Heterogeneity

## Robustness checks

- [ ] Alternative sample
- [ ] Alternative outcome construction
- [ ] Alternative clustering

## Tables and figures planned

1. Summary statistics
2. Main result
3. Heterogeneity
4. Robustness appendix
```

This is a good example of Markdown handling **structure and planning** while math is delegated to LaTeX-style math syntax.

---

65. Markdown Source vs. Rendered Output: Fast Mental Model

You type	Meaning
`# Title`	heading
`**text**`	bold
`*text*`	italic
`` `code` ``	inline code
`- item`	bullet
`1. item`	numbered item
`> quote`	blockquote
`[text](url)`	link
`![alt](img.png)`	image
`---`	horizontal rule
cells separated by vertical bars	table row
`$x^2$`	inline math in math-enabled renderer
`[^1]`	footnote in supporting renderer
`<!-- ... -->`	hidden comment

---

66. Quick Cheat Sheet: Most-Used Syntax

Text

```
**bold**
*italic*
~~strikethrough~~
`inline code`
```

Structure

```
# H1
## H2
### H3
```

Lists

```
- bullet
- bullet

1. first
2. second

- [ ] todo
- [x] done
```

Links / images

```
[link text](https://example.com)
![image description](figures/result.png)
```

Quote

```
> quoted or highlighted text
```

Code

````
```r
summary(df)
```
````

Math

```
Inline: $E[Y_i\mid X_i]$

Display:
$$
\hat\beta = (X'X)^{-1}X'y.
$$
```

---

67. Quick Cheat Sheet: “I Need To...”

I need to…	Use
create a main heading	`# Heading`
create a subsection	`## Heading`
bold an important phrase	`**text**`
italicize a term	`*text*`
show a filename literally	`` `file.csv` ``
show a command literally	`` `\qquad` ``
add a bullet	`- item`
add a numbered step	`1. step`
create a checkbox	`- [ ] task`
insert a link	`[name](URL)`
insert an image	`![alt](path)`
quote text	`> text`
insert a code block	triple backticks
syntax-highlight R code	three backticks followed by `r`
make a simple table	vertical bars between cells, plus a separator row
write inline math	`$...$` in supported renderers
write display math	`$$...$$` in supported renderers
hide a drafting comment	`<!-- comment -->`
add metadata	YAML `--- ... ---`
add a footnote	`[^1]` + definition in supporting renderer
cite from BibTeX	`[@key]` in Pandoc/Quarto
link to a section	`[text](#heading-anchor)`

---

68. Quick Cheat Sheet: Common Mistakes

Symptom	Likely cause	First check
rest of document appears as code	missing closing code fence	count triple backticks
heading not rendered	no space after `#`	write `## Results`
list does not render	formatting/blank-line issue	add blank line before list
image missing	wrong relative path	locate `.md` vs. image
table has extra column	a literal vertical bar inside a cell	escape it with a backslash or rewrite it
math prints literally	renderer lacks math support	check Markdown flavor
citation prints `[@key]`	renderer lacks Pandoc citation support	use Quarto/Pandoc
footnote prints literally	renderer lacks footnote extension	check flavor
paragraph refuses to break	single newline is not paragraph break	add blank line
URL clutters prose	pasted raw URL	use `[name](url)`
underscores trigger formatting	text is not code/escaped	use backticks for variable names

---

69. What Beginners Should Learn First

You do **not** need to memorize the whole guide.

For your first week, learn these ten things:

1. `#`, `##`, `###` headings;

2. blank lines for paragraphs;
3. `**bold**` and `*italic*`;
4. `` `inline code` ``;
5. triple-backtick code blocks;
6. `-` bullet lists;
7. `[text](url)` links;
8. `![alt](path)` images;
9. simple pipe tables;
10. `$...$` math **if your renderer supports it**.

Everything else can be looked up when needed.

---

70. Recommended Beginner Workflow

A practical progression:

Week 1: Use Markdown for seminar notes and paper-reading notes.

Week 2: Create a `README.md` for one research project.

Week 3: Start a dated research log.

Week 4: Put the project under Git and use Markdown for documentation.

Later: Learn Quarto/Pandoc if you want citations, executable code, cross-references, and PDF/HTML output from Markdown.

The goal is not to become a Markdown expert. The goal is to make your research **easier to understand, reproduce, and resume**.

---

71. Final Template: Minimal Markdown Note

Copy this when you need the simplest possible file:

```
# Title

Short description.

## Notes

Write normal paragraphs here.

## Key points

- Point one
- Point two

## TODO

- [ ] Next task
```

---

72. Final Template: Paper Reading Note

```
# Paper Title

**Authors:**
**Year / journal:**
**Link:**
**Date read:**

## Contribution

One sentence.

## Research question

## Data

- Unit of observation:
- Period:
- Sample:
- Main outcome:

## Method / framework

## Main results

1.
2.

## Key assumptions

## What I learned

## Questions

- [ ]

## Connection to my research

## Citation / useful passage

>
```

---

73. Final Template: Research Project README

```
# Project Name

One-paragraph project description.

## Repository structure

```text
project/
|-- code/
```
```

```
|-- data/
|-- figures/
|-- output/
`-- README.md
```

Data

Describe each source and access restrictions.

Replication

1. Run `code/01_clean.R`.
2. Run `code/02_analysis.R`.
3. Run `code/03_output.R`.

Main outputs

- `output/table1.tex`
- `figures/figure1.pdf`

Software requirements

- R:
- Stata:
- Python:

Notes

Document anything a new collaborator would need to know.

---

74. Final Advice

Markdown is intentionally small. Its power comes from combining a few simple conventions with good research habits.

For a researcher, the highest-value uses are usually not elaborate formatting. They are:

- keeping notes readable;
- recording what you did;
- documenting code and data;
- making research folders understandable;
- preserving decisions and unresolved questions;
- creating reproducible project instructions;
- writing source text that remains readable even without rendering.

If a Markdown file becomes difficult to read as plain text, you may be fighting the format. At that point, consider whether LaTeX, Quarto, a spreadsheet, or Word is the better tool.